논문 리뷰: Auto-Encoding Variational Bayes (VAE)

📝 **논문 제목 :** Auto-Encoding Variational Bayes
저자 : Diederik P. Kingma, Max Welling
발표 : ICLR 2014 (arXiv: 1312.6114)
링크 : https://arxiv.org/abs/1312.6114

1. Introduction

Background

생성 모델(Generative Model) : 데이터의 분포 $p(x)$를 학습하여 새로운 샘플을 만들어내는 모델

기존:

- **최대우도추정(MLE)**: 직접적으로 $p(x)$를 모델링하기 어려운 경우가 많음
- **MCMC 계열**: 샘플링이 느리고 고차원에서 비효율적
- **Wake-Sleep 알고리즘**: 학습이 불안정하고 확장성 부족

핵심 문제: 잠재변수(latent variable) z를 도입 시 **사후분포(posterior) $p(z|x)$를 계산하는 것이 intractable**함.

$p(z|x) = \frac{p(x|z)p(z)}{p(x)}$

분모인 $p(x) = \int p(x|z)p(z)dz$ 가 적분 불가능한 경우가 대부분 → 변분 추론 (Variational Inference) 필요

Motivation

1. **효율적 근사 추론**: 복잡한 사후분포를 다루기 쉬운 분포로 근사

2. **확장 가능한 generative 학습**: 대규모 데이터셋에서도 빠르게 학습 가능한 알고리즘 설계

---

2. Related Work

방법	특징	한계	**해결**
Variational Inference	• posterior 분포를 직접 계산하기 어려울 때 이를 근사하기 위한 방법	• variational objective (ELBO)를 최적화하기 위한 gradient 계산이 어려움 • 연속 latent variable을 가지는 모델에서 효율적인 gradient 계산 방법이 제한적 • NN과 결합 힘듦	• reparameterization을 통해 gradient 효율적 계산 → stochastic gradient descent로 최적화 가능 • DL 학습 방식과 연결 • ELBO를 학습가능한 objective로 만듦
Wake-Sleep Algorithm	• generative model과 inference model을 함께 학습하는 방법 • Wake 단계에서는 generative model의 파라미터를 업데이트하고, Sleep 단계에서는 inference model을 업데이트	• Wake-Sleep 알고리즘은 실제 variational lower bound를 직접 최적화 ❌ • gradient 간접 계산 • 안정성 ⬇️	• variational lower bound를 직접 최적화하며, 이론적으로 더 타당한 목적함수 • gradient 직접 계산 → 기반 최적화 가능 • 안정성 ⬆️
Monte Carlo gradient estimation 관련 연구	• Monte Carlo 방법을 이용하여 variational objective의 gradient를 추정하는 연구들	• gradient variance가 매우 큼 → 학습 불안정 → 수렴 안됨 • $z \sim N(\mu, \sigma^2)$	• SGVB estimator는 reparameterization trick을 사용하여 gradient variance를 크게 줄임 • 확률적 sampling을 deterministic transformation으로 변환 • $z = \mu + \sigma\varepsilon$ • $\varepsilon \sim N(0, 1)$
Autoencoder와의 관계	• Autoencoder는 입력 데이터를 latent representation으로 인코딩한 후 다시 복원	• probabilistic interpretation이 부족	• autoencoder 구조를 가지면서도 명확한 probabilistic framework를 제공

방법	특징	한계	**해결**
			• encoder는 posterior 분포를 근사, decoder는 generative model 역할을 수행 • latent variable의 분포를 명시적으로 모델링

흐름

1️⃣ Variational inference

확률 모델 학습 방법 제공

2️⃣ Wake-Sleep

encoder-decoder 공동 학습 아이디어 제공

3️⃣ Monte Carlo gradient

확률적 objective 최적화 방법 제공

4️⃣ Autoencoder

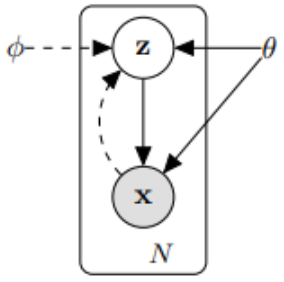
representation learning 구조 제공

---

3. Method

Architecture

VAE는 크게 두 개의 네트워크로 구성




```
입력 x
   |
   ▼
┌──────────────────────┐
│   Encoder (인코더)    │
← 추론 모델 q_φ(z|x)
│   φ (파이) 파라미터   │
└──────────────────────┘
   |                  |
```

```
    ▼                    ▼
   μ(x)              log σ²
(x)
    └──────────────────┘
            │
            ▼
    z = μ + σ·ε   ← Reparam
eterization!
    (ε ~ N(0, I))
            │
            ▼
┌──────────────────────────┐
│    Decoder (디코더)      │
← 생성 모델 p_θ(x|z)
│    θ (세타) 파라미터     │
└──────────────────────────┘
            │
            ▼
        재구성 x̂
```

- **Encoder(inference model)** $q_phi(z|x)$
  - 입력 : x
  - 출력 : 잠재공간의 **평균** μ **와 분산** σ^2
  - encoder는 posterior를 근사
- **Decoder** $p_theta(x|z)$
  - 입력 : latent variable z
  - 출력 : 데이터 분포 $p(x|z)$
  - 원본 데이터를 통해 얻은 데이터 분포로 데이터 생성
- **Prior** $p(z) = mathcalN(0, I)$: 잠재변수의 사전분포는 표준 정규분포

Main Idea: ELBO & Reparameterization Trick

1️⃣ ELBO (Evidence Lower BOund)

직접 $log p(x)$를 최대화하는 대신, 그 **하한(lower bound)**인 ELBO를 최대화:

$$\log p(x) \geq \mathcal{L}(\theta, \phi; x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)]}_{\text{재구성 손실 (Reconstruction Loss)}} - \underbrace{D_{KL}(q_\phi(z|x) \| p(z))}_{\text{KL 정규화 항}}$$

- **재구성 손실**: 인코딩한 z로 x를 얼마나 잘 복원하는가 → 크로스 엔트로피 or MSE
- **KL 발산**: 학습된 잠재 분포가 사전분포 $N(0, I)$에 얼마나 가까운가 → 정규화 역할

KL 항은 가우시안 가정 하에서 **closed-form**으로 계산 가능:

$$D_{KL}(q_\phi(z|x) \| p(z)) = -\frac{1}{2} \sum_{j=1}^{J} \left(1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2\right)$$

2️⃣Reparameterization Trick

문제: $z \sim q_p hi(z|x)$에서 샘플링하면 **역전파(backprop)가 불가능**

해결: 샘플링을 분리하여 미분 가능하게 만들기

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I)$$

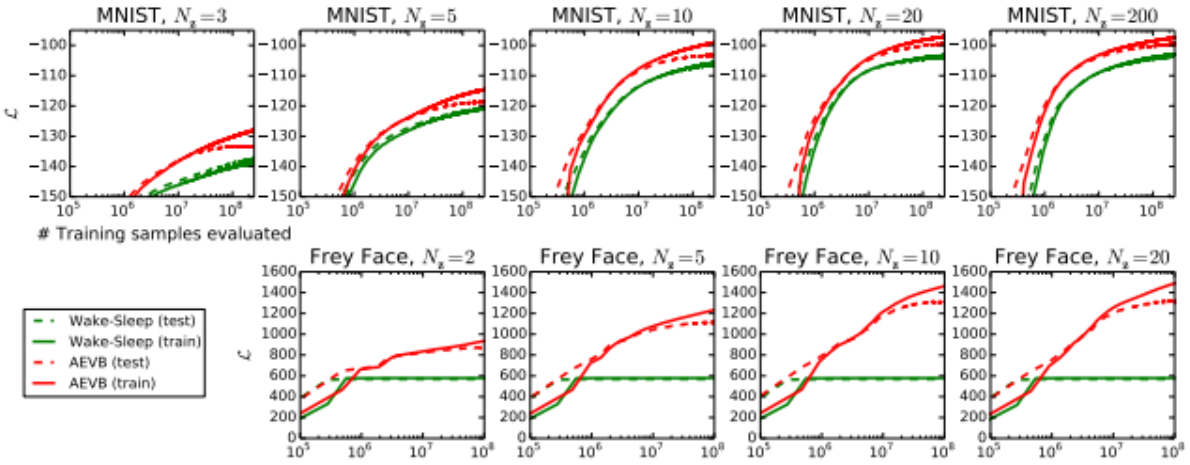
→ 랜덤성(ϵ)을 **외부로 분리**하면, z는 phi에 대해 **미분 가능**!

→ 표준 경사하강법(SGD/Adam)으로 end-to-end 학습 가능

---

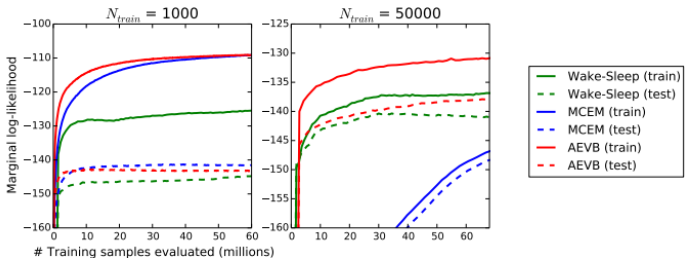
5. Experiments

- **데이터셋**: MNIST 손글씨 데이터셋, Frey Face, CIFAR-10 - classification
- **비교 대상**: Wake-Sleep Algorithm
- **평가 지표**: variation lower bound, the estimated marginal likelihood
- 제안한 SGVB / AEVB (VAE) 방법이 실제로 좋은 generative model을 학습할 수 있는 지 확인
    - decoder(생성모델)은 latent variable z로부터 데이터 생성, encoder(inference model)은 입력 x가 주어졌을 때 latent variable의 posterior 분포 근사
    - 학습 과정에서는 stochastic gradient descent를 사용, SGVB estimator를 통해 variational lower bound 최적화



- 빨간색이 초록색(비교군)보다 더 데이터의 분포를 잘 학습한 것을 볼 수 있음.
- latent 공간이 부드럽게 구성
- latent z를 조금씩 바꾸면 숫자 형태도 자연스럽게 변화
  - latent vector z가 데이터의 중요한 특징 압축, 의미있는 representation 학습
- 모델은 일부 라벨이 있는 데이터 + 라벨이 없는 데이터를 함께 사용해 학습
- latent variable 활용시 데이터 구조 파악 더 유용 → 분류 성능 향상에 도움
- latent variable을 포함한 모델이 기존보다 더 경쟁력 有

→ Variational Inference 방법이 단순한 generative model 뿐만이 아니라 지도학습에도 활용 가능



- toy dataset 뿐만 아니라 복잡한 자연 이미지에도 적용 가능

주요 결과

모델	MNIST (log-likelihood 하한)	Frey Face
Wake-Sleep	≈ -597 nats	≈ -1868 nats
VAE (AEVB)	**≈ -138 nats**	**≈ -1735 nats**

→ VAE가 Wake-Sleep보다 현저히 높은 lower bound 달성

잠재공간 시각화 (2D)

- 잠재 차원 $J=2$로 설정 시 MNIST 숫자들이 **의미 있는 방식으로 군집화**됨
- 비슷한 숫자(4와 9, 3과 8 등)가 잠재공간에서 **인접하게** 위치
- 잠재공간을 격자 형태로 샘플링하면 **연속적으로 변하는 이미지** 생성 가능

생성 품질

- 학습 데이터에 없는 새로운 샘플을 $z sim p(z) = mathcal{N}(0,I)$에서 생성 가능
- Frey Face에서는 표정, 각도 등이 잠재 차원에 인코딩됨

---

6. Conclusion

Contributions

1️⃣ SGVB estimator 제안

- 기존의 Variational Inference : gradient 계산 어려웠음
- SGVB는 reparmeterization trick을 사용해 gradient 계산 가능하게 만듦 → 표준 SGD로 학습 가능

2️⃣ AEVB 알고리즘 제안 (VAE의 핵심 구조)

- encoder network → posterior 근사
- decoder network → generative model
  - 둘을 동시 학습

→ **neural network로 variational inference를 가능하게 만든 방법 제시**

Future Work (후속 연구)

- hierarchical generative model (deep architecture)
  - encoder와 decoder를 더 깊은 신경망으로 확장

    예: CNN encoder, deep latent hierarchy
  - 논문 이후 후속 연구 : Hierachical VAE, Ladder VAE, NVAE

- time-series model (동적 베이지안 모델)
  - latent variable이 시간에 따라 변화
  - 거의 모든 latent variable 모델에 적용 가능한 범용 framework
- global parameter inference
  - vae에서는 주로 datapoint마다 latent variable이 존재하지만 데이터 전체에 공통된 latent parameter 존재 가능, 확장 가능성 ⬆️
- supervised learning with latent variables
  - latent variable을 분류 문제에 적용

→ **probabilistic inference를 deep learning 프레임워크 안으로 도입**

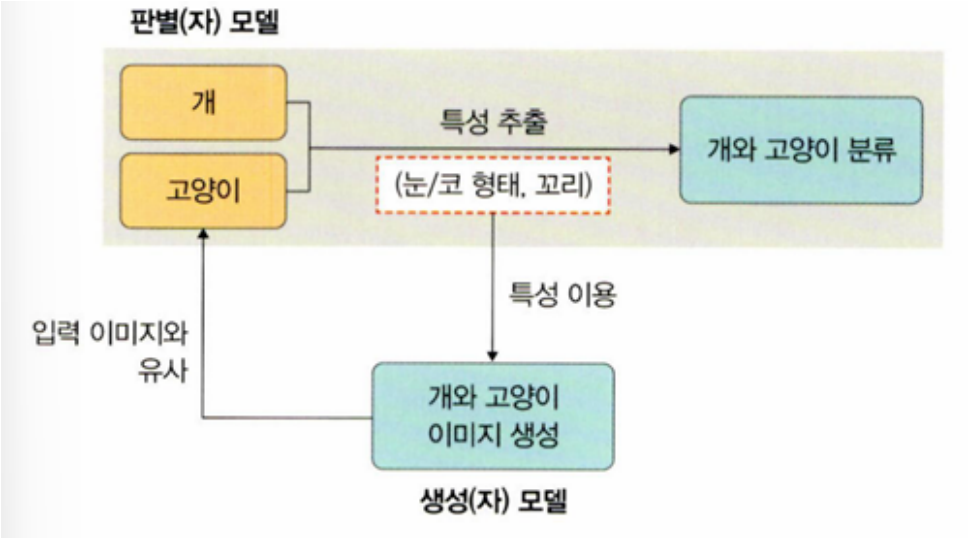
- 이후 발전 흐름 : VAE → β-VAE → VQ-VAE → Diffusion model → Latent diffusion

---

13. 생성 모델

- 주어진 데이터를 학습해 데이터 분포를 따르는 유사한 데이터 생성

13.1.1 생성 모델 개념



- 판별자 모델(discrimiative model) : 분류에 필요한 특성 추출
- 생성자 모델(generative model) : 입력 이미지에 대한 데이터 분포 $p(x)$를 학습해 새로운 이미지 생성

13.1.2 생성 모델 유형

1️⃣ 변형 오토 인코더 - 명시적 방법

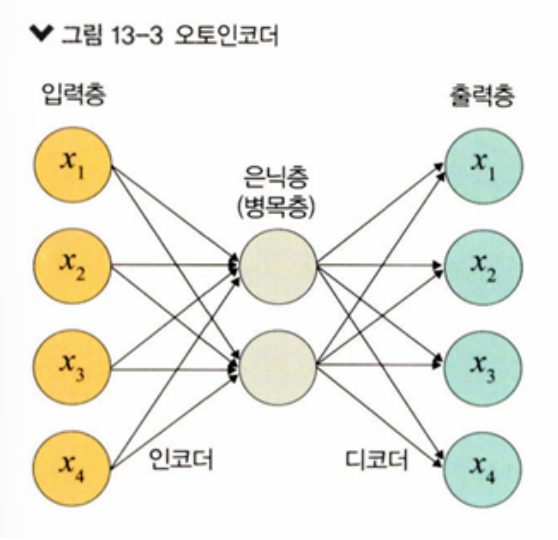
- 확률 변수 $P(X)$ 정의
- 이미지의 잠재 공간에서 샘플링해 완전히 새로운 이미지나 기존 이미지 변형

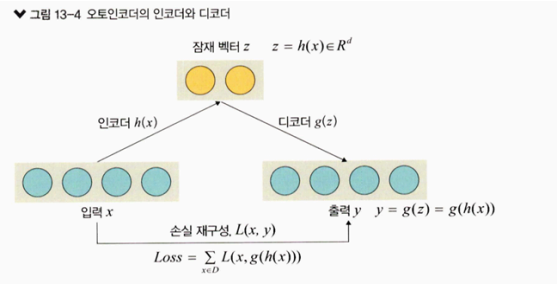
2️⃣ GAN - 암시적 방법

- 확률 변수 $p(x)$ 정의 X, 샘플링해 사용
- 생성자와 판별자가 서로 경쟁하며 가짜 이미지를 진짜 이미지와 최대한 비슷하게 생성하도록 학습

13.2 변형 오토 인코더

13.2.1 오토인코더란





1️⃣ 인코더

- recognition network
- 특성에 대한 학습 수행

2️⃣ 병목층(은닉층)

- 모델 뉴런 개수 최소

1️⃣ 입력 : x

2️⃣ latent vector z

$$z = h(x)$$

3️⃣ 출력 : y

- 가장 차운이 낮은 입력 데이터의 압축 표현 포함

3️⃣ 디코더

- 생성 네트워크
- 압축을 원래대로 재구성 - 최대한 입력에 가까운 출력 생성

4️⃣ 손실 재구성

- 입력층 = 출력층 뉴런 개수 동일
- 압축된 입력을 출력층에서 재구성
- 손실함수는 입력과 출력의 차이로 계산

$$y = g(z) = g(h(x))$$

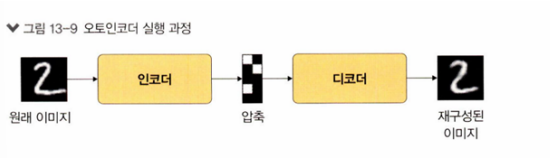
4️⃣ Loss function

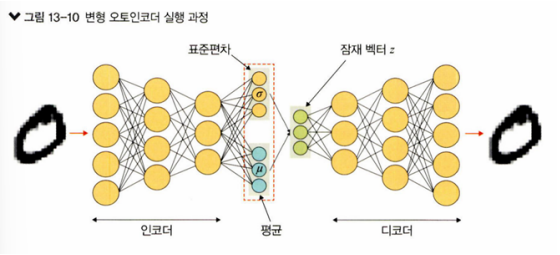
$$Loss = \sum_{x \in D} L(x, g(h(x))) = \| x - y \|^2$$

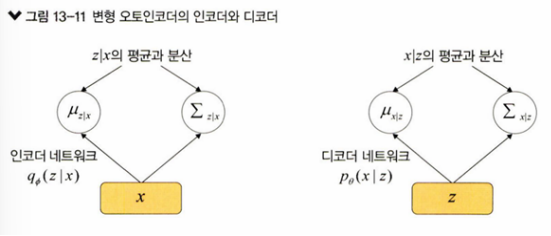
오토 인코더가 중요한 이유 3가지

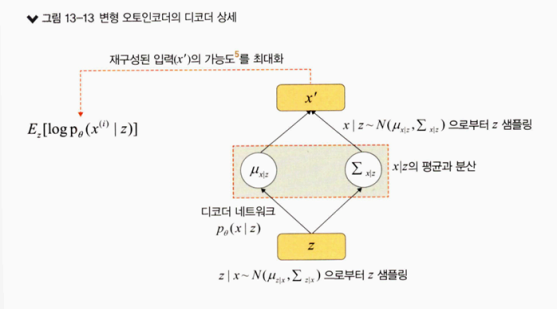
1. 데이터 압축
2. 차원의 저주 예방
3. 특성 추출

13.2.2 변형 오토 인코더

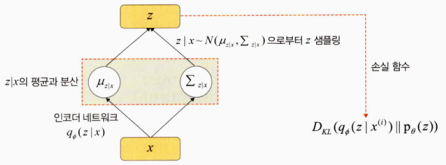








❤ 그림 13-12 변형 오토인코더의 인코더 상세



1. 입력 x를 인코더 네트워크 $q_\phi(z|x)$에 보내 $(\mu_{z|x}, \Sigma_{z|x})$를 출력하고, 이를 이용하여 다음 수식의 ②항에 대한 값을 구합니다.

$$L(x^{(i)}, \theta, \phi) = \underbrace{E_z[\log p_\theta(x^{(i)} \mid z)]}_{①} - \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,\|\, p_\theta(z))}_{②}$$

2. $(\mu_{z|x}, \Sigma_{z|x})$의 가우시안 분포에서 z를 샘플링합니다.

$$z \leftarrow z \mid x \sim N(\mu_{z|x}, \Sigma_{z|x})$$

1. 샘플링한 z를 디코더 네트워크 $p_\theta(x|z)$에 보내 $(\mu_{x|z}, \Sigma_{x|z})$를 출력한 후 이를 이용하여 ①항의 값을 구합니다(다음 수식은 앞의 수식과 동일합니다).

$$L(x^{(i)}, \theta, \phi) = \underbrace{E_z[\log p_\theta(x^{(i)} \mid z)]}_{①} - \underbrace{D_{KL}(q_\phi(z \mid x^{(i)}) \,\|\, p_\theta(z))}_{②}$$

2. $(\mu_{x|z}, \Sigma_{x|z})$의 가우시안 분포에서 z를 샘플링한 후 x'를 구합니다.

$$x' \leftarrow x \mid z \sim N(\mu_{x|z}, \Sigma_{x|z})$$

3. 역전파를 이용하여 $L(x^{(i)}, \theta, \phi)$의 값이 높아지는 방향으로 기울기를 업데이트합니다. 즉, 가능도(likelihood)가 증가하는 방향으로 파라미터 θ와 ϕ를 업데이트합니다.